

Claude Chat History

1791388240128_20_mastermind.zip

1791388240128_20_mastermind.zip

ZIP

Scenario 20 — Mastermind

A terminal code-breaking game with repeated symbols and structured feedback.

Provided files

- `main.py` — entry point.
- `game.py` — game loop and history.
- `logic.py` — exact and partial match calculation.
- `requirements.txt` — dependency declaration.

Setup

`python main.py`

Before changing the code

Construct codes and guesses containing repeated symbols. Reason about how many times a single code occurrence should be allowed to contribute to feedback.

Task 1 — Correct duplicate-aware feedback

Implement feedback so each code position can be counted at most once. Exact matches must be resolved before partial matches.

Done when: repeated-symbol cases produce the correct exact and partial counts.

Task 2 — Complete game lifecycle

Add clear win/loss handling, preserve guess history, enforce the guess limit, and prevent state changes after the game ends.

Task 3 — Difficulty modes

Add difficulty choices that vary code length, symbol range, and allowed guesses. The feedback algorithm must work for all supported configurations.

Task 4 — History and input robustness

Present a readable guess history and reject malformed guesses without consuming turns. Feedback must correspond to one accepted guess only.

Required testing

Test exact matches, repeated symbols, no matches, mixed exact/partial cases, every difficulty, invalid guesses, final-turn wins, losses, and quitting.

LLM usage

You may use an LLM during the lab. The goal is to use it as a coding assistant while retaining responsibility for understanding and testing the result.

- Inspect the existing code before asking for changes.
- Ask for explanations when you do not understand a proposed change.
- Test generated code against the stated behaviour and edge cases.
- Keep your complete LLM chat history for submission.

- Do not replace the whole project with an unrelated implementation.
- Keep all state in memory; do not add CSV, JSON, SQLite, or other persistence.

Submission checklist

- Task 1 completed and the original defect was reproduced and fixed.
- Tasks 2–4 completed and tested.
- Boundary and invalid-input cases tested.
- No unnecessary external dependencies added.
- No persistent storage added.
- Code remains understandable and modular.
- Complete LLM chat-history link included.

Folder structure

scenario-08-mastermind/

└─ README.md

└─ requirements.txt

└─ main.py

└─ game.py

└─ logic.py

Submission Checklist

Submission is only the following three things:

- A 10-second video of gameplay before your changes, showing the bug/broken behavior
- A 10-second video of gameplay after your changes, showing the bug fixed and the new features working
- The Chat/LLM used page link, with the complete chat history

let us do it task by task.

we shall now do task 1.

give me only the edited files and the reason why you did so and so edits in the code

Ran 5 commands, shared files

Only logic.py needed to change for Task 1. main.py and game.py are untouched.

The original defect

The old code did this:

python

```
exact = sum(a == b for a, b in zip(code, guess))
```

`partial = sum(ch in code for ch in guess) - exact`

`ch in code` only asks whether the symbol exists anywhere in the code, so one code position can be reused for every matching guess symbol. For code 1234 and guess 1111, the correct feedback is (1, 0). The old code returned (1, 3), because each of the three extra 1s in the guess "found" the single 1 in the code. In total, 5 of my 13 test cases failed on the original code.

The file also had a broken comment at the top, which was a leftover fragment ("the code, so duplicate symbols can consume...").

What I changed and why

1. **Pass 1 resolves exact matches and consumes those positions.** I loop over `zip(code, guess)`. If the symbols match, I count an exact. If not, I put the code symbol and guess symbol into `unmatched_code` and `unmatched_guess`. This follows the task rule that exact matches are resolved first, and it removes exactly-matched positions from consideration on both sides.
2. **Pass 2 pairs up only the leftovers, using Counter and min.** For each symbol, `min(code_count, guess_count)` is how many times it can be paired. This is what enforces that each code position is counted at most once. Counter comes from the standard library, so no new dependency is added.
3. **Length check.** feedback now raises `ValueError` if code and guess have different lengths, instead of `zip` silently truncating. This prepares for Task 3 (variable code length) and surfaces a bug early if the game ever passes mismatched lists.
4. **Docstring and comments** replace the broken comment, explaining what exact and partial mean and why the two-pass approach is used.
5. **Line endings.** I kept the project's original CRLF line endings so your git diff shows only the real changes.

Testing

- **Defect reproduced.** The original code failed 5 of 13 cases, for example 1234/1111 gave (1, 3) instead of (1, 0).
- **Fixed code passes all 13.** The cases cover all-exact, no matches, all-partial, mixed, swapped pairs, extra repeats in the guess, repeats in the code, and a 6-length code.
- **Independent cross-check.** I wrote a separate brute-force reference that crosses positions off one by one, and compared it on 401,052 code/guess pairs across several lengths and symbol ranges. It found 0 mismatches.
- **Smoke test.** The game still runs end-to-end.

Three of my own hand-written expected values were wrong at first (1213/1121, 11223/12121, 1234/2111), and I corrected them by working through each case position by position. That is why the independent reference is useful: for the "before" video, use 1234 vs 1111 or 1123 vs 1111.

Send me Task 2 when you're ready.

Logic

PY

let us now focus on task 2 and explain me the changes you made too. so that i can understand it well

Designing the game state class and guess-handling flow.

Let me start from the current project state and check how the existing game.py behaves before changing it.

Ran 4 commands, shared files

Only game.py changed for Task 2. logic.py still has the Task 1 fix, and main.py is untouched.

The problem with the original

All the rules lived inside one run() loop, with loose fields (turns, history). The game had no concept of a status like "won" or "lost", and nothing could stop a guess after the game ended. It only worked because the loop happened to return.

The new design makes the rules part of the game object itself, so they hold no matter who calls it.

The changes

1. A status field with four states (PLAYING, WON, LOST, QUIT).

This is the core of Task 2. The game is always in exactly one state, and is_over simply means "status is not PLAYING". Win, loss and quit are now clear, separate outcomes instead of three different return statements.

2. turns_left is derived from the history.

Before, turns and history were two separate things that had to be kept in step by hand. Now `turns_left = max_turns - len(history)`. Every accepted guess adds one history entry, so the turn count cannot drift out of sync with the history. It also guarantees that the guess limit is enforced and that history is preserved.

3. A new method, make_guess(raw), is the only way to play a turn.

It does these steps in a fixed order:

1. If the game is over, raise `GameOverError`. This is what prevents state changes after the game ends.
2. Validate the guess. If it's malformed, raise `ValueError`.
3. Compute feedback and append it to the history.
4. Update the status.

The checks come before anything is modified, so a rejected guess leaves the game completely unchanged. That is why invalid input never costs a turn.

4. The win check comes before the loss check.

python

```
if exact == len(self.code): self.status = WON
```

```
elif self.turns_left == 0: self.status = LOST
```

If I tested "out of turns" first, a correct guess on the final turn would count as a loss. The order of those two lines is what makes final-turn wins work.

5. quit() does nothing once the game has ended.

Without that guard, quitting after a win could overwrite WON with QUIT.

6. run() is now only the terminal interface.

It reads input, calls make_guess, catches ValueError and GameOverError, and prints. The rules are not in it any more. This makes the logic testable without typing into a terminal, and it keeps the code modular, as the checklist asks.

7. show_result() prints a distinct message per outcome.

You get "Cracked the code in N guesses", "Out of turns. You lose", or "You quit", and each one reveals the code.

8. Mastermind(code=None, max_turns=10) accepts optional arguments.

This lets tests use a known secret like "1234" instead of fighting randomness. Calling Mastermind() with no arguments plays exactly as before.

9. EOFError handling.

If the input stream closes (for example Ctrl-D), the game ends as a quit instead of crashing with a traceback. This is a small addition beyond the task text.

10. Constants at the top (CODE_LENGTH, SYMBOLS, MAX_TURNS).

The numbers 4, "123456" and 10 were scattered through the old code. Putting them in one place sets up Task 3, where difficulty will just change these values.

Testing

I ran 39 checks, all passing. They cover:

- Win, loss, quit, and a final-turn win, plus a final-turn wrong guess with one turn allowed.
- A guess after a win, loss or quit raises GameOverError, and the history, turns and status are identical before and after.
- A correct guess after a loss cannot flip the result to a win, and quit() after a win can't overwrite it.
- Seven kinds of malformed input are rejected without consuming a turn.
- Full run() play-throughs with simulated input, including uppercase Q and Ctrl-D.
- Two real terminal runs, one quitting and one losing.

Two of my tests failed at first because of mistakes in the tests themselves, not the game. I was checking prompt text that a mocked input() never prints, and one assertion had an or that could have hidden a bug. I fixed both before the final run.

Some behaviour to expect:

- Because validation now lives in make_guess, you will see it listed there. Task 4 will build on it by improving the error messages and the history display.
- The game is still fixed at 4 digits from 1 to 6, because variable settings are Task 3.

Send me Task 3 when you're ready.

Game

PY

Ran 2 commands

All four planted bugs are caught, so the tests can detect real regressions. Now I'll verify every value in the manual test table by running it, so nothing I hand you is from memory.

Ran a command, shared files

I've written an automated test file you can run yourself, plus a manual checklist with the expected answers. I also owe you the Task 4 explanation and the partial-match explanation from last time.

How partial is calculated

Partial means "right symbol, wrong place", where each code position can be used only once. It happens in two passes:

1. **Exact pass.** Go position by position. If `code[i] == guess[i]`, count one exact, and **remove both positions from play**. Everything else goes into two leftover piles.
2. **Partial pass.** For each symbol, partial = the smaller of (how many times it's in the leftover code) and (how many times it's in the leftover guess). Add those up across symbols.

Worked example. Code 1123, guess 1111:

- Exact pass: positions 0 and 1 match, so exact = 2. Leftover code is 2 3 and leftover guess is 1 1.
- Partial pass: the leftover code has no 1s, so partial = 0. The result is **(2, 0)**.

Another one. Code 1234, guess 2111:

- Exact pass: nothing matches in place. Leftover code is 1 2 3 4 and leftover guess is 2 1 1 1.
- Partial pass: for 1, $\min(1 \text{ in code}, 3 \text{ in guess}) = 1$. For 2, $\min(1, 1) = 1$. The result is **(0, 2)**.

These are the things people most often find confusing:

- Partial is **not** "how many guess symbols appear somewhere in the code". That was the original bug.
- Extra copies of a symbol in your guess score nothing once the code's copies are used up.
- A symbol that scored as exact can't also score as partial.

You can sanity-check any answer by hand: **exact + partial must equal the sum, over each symbol, of $\min(\text{count in code}, \text{count in guess})$** . For 1234 vs 2111, that's $\min(1,3) + \min(1,1) = 2$, and $0 + 2 = 2$. ✓

What Task 4 changed (game.py)

- **Specific error messages.** The game now says what is wrong: empty guess, wrong length with the count you entered, or the exact bad characters, for example Invalid character(s): 'a', 'b'. Before, everything got the same generic message.
- **History table.** `format_history()` builds an aligned table, shown after each accepted guess. Typing h or history shows it for free.

- **Feedback is tied to one accepted guess.** `make_guess` validates before computing anything, so a rejected guess never reaches `feedback()`. The history gets exactly one row per accepted guess.
- **Non-text input** (like `None` or a number) raises `ValueError` instead of crashing.

The automated tests

Run these from your project folder. The file uses only the standard library:

```
python -m unittest -v test_mastermind
```

It has 30 tests covering all four tasks: feedback tables and a cross-check against a slow reference, every difficulty, win/loss/quit, final-turn wins, no changes after the game ends, 20 kinds of invalid input, history, and scripted terminal runs. I also planted four bugs in a temporary copy to prove the suite catches them: the original Task 1 defect, a swapped win/loss order, invalid guesses consuming a turn, and a missing game-over guard. All four were caught.

This file is optional extra evidence. Your submission is still the two videos and the chat link.

Manual test cases

The secret is random, so to test with a known one, run the game like this:

```
python -c "from game import Mastermind; Mastermind(code='1234').run()"
```

For hard mode, use `Mastermind(code='12345', difficulty='hard')`. I ran every row below to confirm the expected values.

Feedback, with code 1234:

Guess	Expected	Why
1111	Exact 1, Partial 0	One 1 in the code, used by the exact match
2111	0, 2	2 and one 1 match, the other 1s get nothing
4321	0, 4	All partial
1243	2, 2	Mixed
5566	0, 0	No matches
1234	win	

Repeats in the code, with code 1122:

Guess	Expected
2211	0, 4
1212	2, 2
1111	2, 0
2222	2, 0
1221	2, 2

Invalid input, in medium mode. Each should print a message, show the same turn count, and add no history row:

Type	Expected message
(just Enter)	Empty guess...
123	Your guess has 3 digits...
12345	Your guess has 5 digits...
abc	Invalid character(s): 'a', 'b', 'c'...
1 34	Invalid character(s): ' '...
7777 or 0000	Invalid character(s): '7' / '0'

Lifecycle:

- **Final-turn win.** Play a fixed code with a wrong guess on every turn except the last, then guess it right. You should get "Cracked the code", not "You lose".
- **Loss.** Use up all the guesses. You should get "Out of turns" and the code revealed.
- **Quit.** q, Q and Ctrl-D should each give "You quit".
- **h at the very start** should print No guesses yet. and cost no turn.

Difficulty boundaries:

- On easy, 1235 is rejected and 4444 is accepted.
- On hard, 4-digit guesses are rejected, 8 is accepted and 9 is rejected.
- At the menu, 1/2/3, hard and HARD work, while 0, 4, blank and banana are re-asked.

For your "before" video, use the original code with secret 1234 and guess 1111. It will show Exact 1, Partial 3 instead of 1, 0.

Test mastermind

PY

the code works well

Confirming submission details alongside the working code.

Confirming submission details alongside the working code.

Glad it's working. Here's what's left for your submission:

- **"Before" video (about 10 seconds):** Run the original code from your zip and use the bug case. Guess 1111 against a code with a single 1 in it, such as 1234, and the original shows Exact 1, Partial 3 instead of 1, 0. Since the secret is random in the original, you can just keep guessing repeated digits like 1111, 2222 until the partials look inflated.
- **"After" video (about 10 seconds):** In the fixed version, show the difficulty menu, a few rejected guesses (abc, 12345) that don't cost a turn, the history table growing, and a win or loss message.
- **Chat link:** Use the share link for this whole conversation so the full history is included.

The folder structure in your brief lists a README.md. If you'd like one, I can write a short README covering how to run the game, the difficulty modes, the commands (h and q), and how to run the tests.